

Politechnika Gdańska

Katedra Optoelektroniki i Systemów Elektronicznych

Organizacja i Oprogramowanie Systemów Elektronicznych

Michał Kowalewski

Programowanie w środowisku MS Visual C++



Politechnika Gdańska
2010

1. Wprowadzenie

Celem ćwiczenia jest zapoznanie się ze środowiskiem Microsoft Visual Studio 2008 i napisanie prostego programu opartego o okna dialogowe.

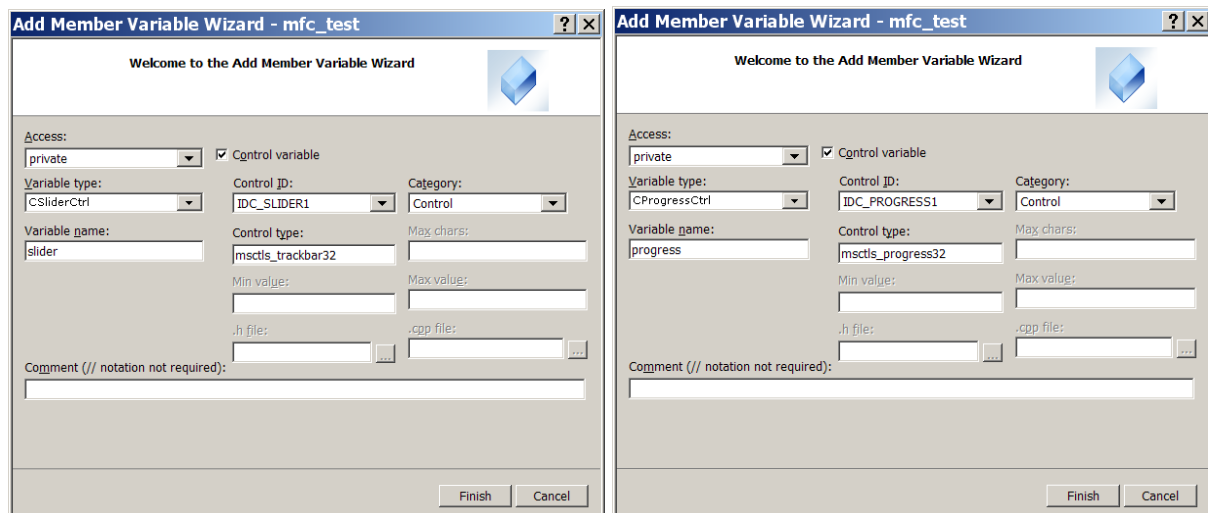
Programowanie realizowane jest w języku C++.

2. Zadania do wykonania

1. Uruchomić środowisko MS Visual Studio 2008
2. W menu **View** wybrać następujące okna: *Solution Explorer*, *Class View*, *Resource View*, *Output*, *Toolbox*, *Properties Window*. Pozostałe okna zamknąć.
3. Utworzyć nowy projekt bazujący na bibliotece MFC i oparty o okna dialogowe:
 - a. Uruchomić kreatora aplikacji:
File/New/Project → Project Types/Visual C++/MFC/MFC Application
 - b. W polu **Name** wpisać nazwę programu i kliknąć OK.
 - c. W kolejnych krokach wybrać rodzaj aplikacji: **Dialog Based**.
4. Wstawić na panelu dwie kontrolki: *Slider Control* i *Progress Control*.
5. Zdefiniować w klasie okna dialogowego Cnazwa_programuDlg zmienne kontrolne **slider** i **progress** przypisane kontrolkom *Slider Control* i *Progress Control*.

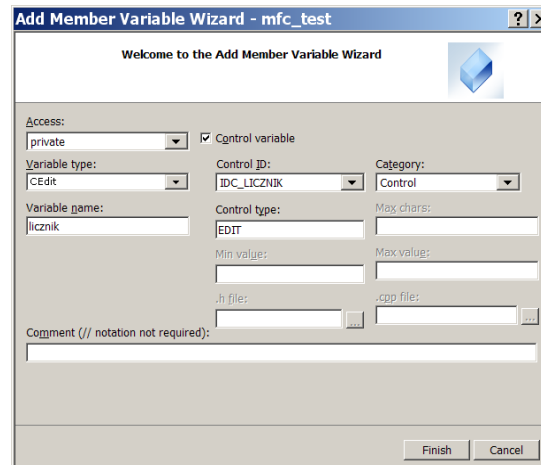
Procedura przebiega następująco:

- a) wybrać kontrolkę na panelu
- b) z menu kontekstowego wybrać opcję **Add Variable**
- c) ustalić parametry zgodnie z rysunkami



6. Dodać do programu procedurę obsługi zdarzenia zmiany położenia suwaka kontrolki *Slider Control*. W tym celu z menu kontekstowego wybrać opcję **Add Event Handler**. Wybrać komunikat powiadomienia kontrolki NM_CUSTOMDRAW i wybrać **Add and Edit**,
7. Uzupełnić treść funkcji dodając kod `progress.SetPos(slider.GetPos());` ;
Sprawdzić działanie programu.

8. Wstawić na panelu trzy przyciski **Button** oraz pole edycyjne **EditControl**.
9. W oknie **Properties** ustawić parametry kontroltek:
 Button1 → ID = IDC_START → Caption = Start
 Button2 → ID = IDC_STOP → Caption = Stop
 Button3 → ID = IDC_RESET → Caption = Reset
 EditControl → ID = IDC_LICZNIK
10. Dodać zmienną o nazwie **licznik** do kontrolki **EditControl** (opcja **Add Variable**)



11. Zdefiniować w klasie okna dialogowego zmienną prywatną typu **int** o nazwie *counter* oraz zmienną typu **LPTSTR** o nazwie *lpzStringBuf*;
12. Dodać do zasobów aplikacji identyfikator timera ID_TIMER i nadać mu wartości równą 1.
 W tym celu w zakładce **Resource View** wybrać plik *.rc i z menu kontekstowego wybrać **Resource Symbols**. Dodać nowy identyfikator i wpisać **Name: ID_TIMER, Value: 1**.
13. Korzystając z zakładki **Properties/Control Events** dodać funkcje obsługi zdarzenia BN_CLICKED przycisków **Start**, **Stop** i **Reset**. W kodzie programu powinny pojawić się funkcje **OnBnClickedStart**, **OnBnClickedStop** i **OnBnClickedReset**.
14. W oknie **Properties** panelu programu dodać procedury obsługi zdarzeń WM_TIMER i WM_DESTROY. W kodzie programu powinny pojawić się funkcje **OnTimer** i **OnDestroy**.
15. Uzupełnić kod programu (trzykropek oznacza nazwę programu):

```
void C...Dlg::OnBnClickedStart() {
    SetTimer(ID_TIMER, 100, NULL); //Uruchomienie timera
    GetDlgItem(IDC_START)->EnableWindow(FALSE);
    GetDlgItem(IDC_STOP)->EnableWindow(TRUE);
}
void C...Dlg::OnBnClickedStop() {
    KillTimer(ID_TIMER); //Zatrzymanie timera
    GetDlgItem(IDC_STOP)->EnableWindow(FALSE);
    GetDlgItem(IDC_START)->EnableWindow(TRUE);
}
void C...Dlg::OnBnClickedReset() {
    counter=0;
    swprintf(lpzStringBuf,20,L"Licznik = %d",counter);
    licznik.SetWindowTextW(lpzStringBuf);
}
```

```

void C...Dlg::OnTimer(UINT_PTR nIDEvent) {
    // Zwiększenie wartości licznika
    counter++;
    swprintf(lpszStringBuf,20,L"Licznik = %d",counter);
    // Wpisanie wartości licznika do zmiennej licznik
    licznik.SetWindowTextW(lpszStringBuf);
    CDialog::OnTimer(nIDEvent);
}
BOOL C...Dlg::OnInitDialog() {
    CDialog::OnInitDialog();
    // ...
    // TODO: Add extra initialization here
    lpszStringBuf = (LPTSTR)GlobalAlloc(GPTR, 50);
    // Zerowanie licznika
    counter = 0;
    // Dezaktywacja przycisku Stop i kontrolki Licznik
    GetDlgItem(IDC_STOP)->EnableWindow(FALSE);
    GetDlgItem(IDC_LICZNIK)->EnableWindow(FALSE);
    swprintf(lpszStringBuf,20,L"Licznik = %d",counter);
    licznik.SetWindowTextW(lpszStringBuf);
    return TRUE;
}
void C...Dlg::OnDestroy() {
    CDialog::OnDestroy();
    // TODO: Add your message handler code here
    GlobalFree(lpszStringBuf);
}

```

Przetestować program

Zmiana interwału czasowego timera za pomocą okna konfiguracji

1. Dodać do zasobów programu okno dialogowe i zdefiniować klasę tego okna (**konf**) dziedziczną z klasy **CDialog**.
2. Umieścić w oknie jedno pole edycyjne.
3. Dodać do głównego pliku źródłowego programu plik nagłówkowy "konf.h".
4. Dodać do pliku z klasą głównego okna dialogowego zmienną obiektową typu **konf** o nazwie **konfiguracja**.
5. Dodać do programu dwie zmienne globalne typu 'int' (z poziomu zakładki **Class View**):
 - a. **interval** (zmienna podawana w miejsce parametru 100 funkcji **SetTimer**)
 - b. **intervalTemp** (zmienna tymczasowa przechowująca wartość interwału z kontrolki **EditControl** okna konfiguracyjnego, wykorzystywana jeśli naciśnięto **OK**).

Zmienne należy ponadto zadeklarować w głównym pliku nagłówkowym.

6. Dodać do klasy okna konfiguracji funkcję obsługującą zdarzenie **WM_DESTROY**, aby przechwycić wartość z kontrolki **Edit** i wpisać wartość liczbową do zmiennej **intervalTemp**. Skorzystać z funkcji **GetDlgItemInt**.

7. Dodać w głównym oknie programu przycisk uaktywniający okno konfiguracyjne. Kod funkcji wykonywanej przy naciśnięciu przycisku powinien mieć postać:

```
void C...Dlg::OnBnClickedButton1() {
    int result = konfiguracja.DoModal();
    if (result == IDOK) {
        interval = intervalTemp;
    }
}
```

8. Udoskonalić program tak, aby bieżąca wartość zmiennej **interval** pojawiała się w polu edycyjnym automatycznie po wyświetleniu okna konfiguracyjnego.

Wskazówki:

- a. Dodać funkcję **OnInitDialog()**. Funkcję tą należy zadeklarować jako wirtualną

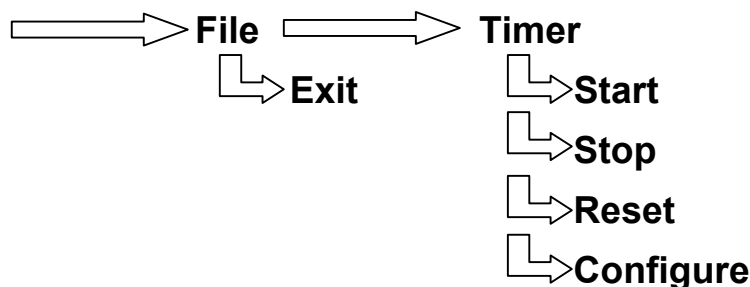
```
virtual BOOL OnInitDialog();
```

⇒ zobacz następną stronę

- b. Skorzystać z funkcji **SetDlgItemInt**.

Utworzenie i oprogramowanie struktury menu programu

1. Dodać do zasobów programu **Menu** posiadające następującą strukturę:



Dla poszczególnych opcji Menu powinny się pojawić w pliku „Resource.h” następujące identyfikatory:

```
ID_TIMER_START, ID_TIMER_STOP, ID_TIMER_RESET, ID_TIMER_CONFIGURE,
ID_FILE_EXIT
```

- W edytorze Menu wygenerować szkielety funkcji callback dla każdej opcji. W tym celu wybrać z menu kontekstowego opcję **Add Event Handler...** Funkcje powinny zostać dołączone do klasy okna dialogowego **C...Dlg**.
- Uzupełnić kod wygenerowanych funkcji callback w ten sposób, aby uzyskać jednakowy efekt pracy programu niezależnie od tego czy wybieramy opcję z Menu, czy też korzystamy z przycisków umieszczonych na panelu.

Zadanie dodatkowe

Rozbudowa programu poprzez dodanie w oknie konfiguracji przycisków wyboru kierunku pracy licznika (Up – zliczanie w górę, Down – zliczanie w dół do zera).

- Dodać w oknie konfiguracji dwa przyciski opcji (Radio Button) oraz nadać im identyfikatory **ID_UP** oraz **ID_DOWN**.
- Wstawić na panelu ramkę grupującą Group Box w taki sposób aby zawierała w sobie przyciski Up i Down.
- W oknie właściwości dla przycisku **Up** zmienić wartość właściwości Group na **True**.

4. Zdefiniować dwie zmienne globalne typu `int`: **direction** i **directionTemp**
5. W kodzie funkcji **DoDataExchange** klasy **konf** dodać wywołanie funkcji **DDX_Radio**
`DDX_Radio(pDX, ID_UP, directionTemp);`
6. Uzupełnić kod źródłowy funkcji **OnInitDialog** i **OnDestroy** klasy **konf**

```

BOOL konf::OnInitDialog() {
    CDialog::OnInitDialog();
    directionTemp = direction;
    UpdateData(FALSE);
    SetDlgItemInt(IDC_EDIT1, interval);
    return TRUE;
}

void konf::OnDestroy() {
    CDialog::OnDestroy();
    // TODO: Add your message handler code here
    intervalTemp = GetDlgItemInt(IDC_EDIT1);
    UpdateData(TRUE);
}

```

7. Dodać w kodzie w kodzie źródłowym funkcji **OnInitDialog** klasy **C...Dlg** inicjalizację zmiennej **direction** wartością **0**.
8. Uzupełnić kod źródłowy funkcji **OnBnClickedKonf** klasy **C...Dlg**

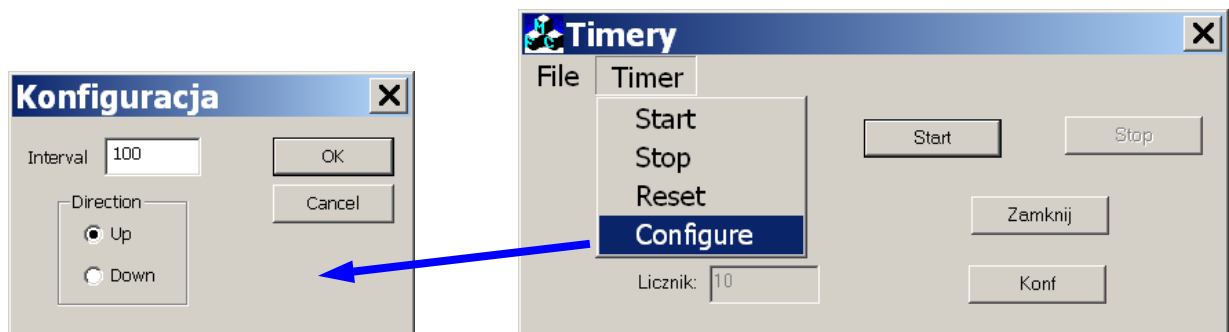
```

void C...Dlg::OnBnClickedButton1() {
    // TODO: Add your control notification handler code here
    int result = konfiguracja.DoModal();
    if (result == IDOK) {
        interval = intervalTemp;
        direction = directionTemp;
    }
}

```

9. Zmodyfikować kod źródłowy funkcji **OnTimer**, aby uzależnić kierunek pracy licznika od wartości zmiennej **direction**.

Efekt końcowy



Przetestować program i zaprezentować prowadzącemu 😊